ChatGPT

TaskFlow AI: Windows Automation Assistant Blueprint

App Name: *TaskFlow AI* – a name evoking seamless "flow" through desktop tasks via AI. It suggests an intelligent assistant that automates workflows on Windows by "flowing" from one task to the next, much like a guided conveyor belt of actions.

Tech Stack & Architecture

- **Desktop Framework: C# on .NET 8** (or latest stable) with **WinUI 3** for the front-end. WinUI 3 is Microsoft's newest native Windows UI framework, optimized for beautiful, fluid desktop apps [1]. It uses XAML for layouts and automatically respects Windows themes (light/dark), DPI scaling, and multi-monitor setups [2].
- **UI Components:** Use a **WebView2** control (Edge engine) to render chat messages in rich Markdown [3]. This allows code blocks, tables, and formatted text to display exactly (selectable, copyable, searchable by Ctrl+F) [3] [4]. Other UI elements (sidebar, input box, menus) use standard WinUI/XAML controls for native look and feel [2].
- **AI Backend: Python + FastAPI** for the cloud service. FastAPI is a popular high-speed Python web framework well-suited to small AI backends [5]. Using Python gives access to AI libraries, and FastAPI has minimal boilerplate and good async support [5]. The backend will expose REST endpoints (e.g. JSON APIs) that the desktop app calls.
- **AI Model: OpenAI GPT-4** (via ChatGPT/Completions API). GPT-4 (or GPT-4o if available) currently leads in conversational quality. The FastAPI backend will forward user prompts to the OpenAI API and return structured responses. (Alternative providers like Anthropic/Claude or open-source LLMs could be considered, but GPT-4 is chosen here for accuracy and ease of integration.)
- **Hybrid Cloud/Local:** The app sends user queries to the cloud for "planning" (natural language understanding and instruction generation) but executes all file, system or network commands locally on the PC. This keeps system control local for security and responsiveness, while leveraging the cloud LLM for intelligence. The local app can parse the AI's JSON outputs to decide on actions.
- **Deployment:** Host the Python/FastAPI service on any cloud or VPS. (FastAPI's docs note that virtually any cloud provider can be used [6].) For example, deploy in a Docker container on AWS or Azure.

UI Structure & Behavior

- The app window has a chat-style layout: a sidebar (left) listing past conversations or quick actions; the main pane showing the conversation; and a bottom text box for user input. It mimics popular chat UI (like ChatGPT) but is more compact and native.
- **Conversation View:** The AI and user messages render in a scrollable central pane. The assistant's replies (markdown via WebView2) can include code boxes, lists, and clickable hyperlinks. Text is fully selectable (so users can copy commands or code) [3] [4]. Buttons or drop zones can allow dragging a file or selecting an app to refer to in chat.

- **Sidebar & Tools:** The sidebar shows past chats or “Personalities” (custom system prompts). There may be buttons/menu items for Settings, billing info, and offline mode. The UI adapts to the Windows system theme (dark/light) and accent color by default [2], so it always looks consistent with Windows 11/10.
- **Automation Interface:** When the AI generates an action plan (e.g. “Move file X to folder Y”), the app displays these as structured steps. The user can review and **confirm** any action before it runs, for safety. We implement a JSON schema for AI outputs so the app can parse actions reliably. (This avoids “hallucination”; if the AI strays from the schema, the app can ask it to re-format.)
- **Sample Flow:** User: “Organize my Desktop screenshots folder.” Assistant replies with JSON: `{"actions":[{"type":"move_file","src":"C:\\Users\\Me\\Desktop\\Screenshots\\A.png","dest":"..."}]}`. The UI shows these actions in human-friendly form (“Move A.png to Archive folder”). User clicks “Execute” to run them.

Boot & Offline Operation

- **Auto-Start on Boot:** The app installs a Windows Scheduled Task set to trigger at system startup [7]. (Alternatively, a registry “Run” entry or Windows Service could be used, but Task Scheduler is simple and reliable [7].) On startup it can run minimized to the tray or background until the user opens it.
- **Offline Detection:** The app constantly monitors internet connectivity. It subscribes to Windows network events and also periodically pings a known server (e.g. google.com) for confirmation [8]. If ping fails consistently, it flags “offline mode.” This hybrid approach (event + ping) is recommended to avoid false positives [8].
- **Offline Fallback:** When offline, the chat interface remains available but the assistant switches to a limited mode. For example, local canned responses or pre-downloaded knowledge can handle simple queries (like looking up local files or help topics). Crucially, any features requiring the cloud (LLM responses) are disabled with an “offline” warning. The app could automatically queue any pending commands for later execution.
- **Action on Disconnect:** Upon losing connection, the app can auto-launch (if not already open) and notify the user: e.g. a pop-up “You’re now offline – please retry tasks requiring the AI.” Optionally, it could trigger a user-defined local task (for instance, pausing backups, switching network adapters, etc.), as configured by the user. (Task Scheduler could also be configured for “On disconnect” triggers, or the app itself can detect and act.) The key is that the assistant remains running offline, ready to execute any already-in-chat commands.

Unique Features vs. Competitors

- **Native Windows Integration:** Unlike generic chatbots or web services, this is a fully native WinUI desktop app that *respects Windows UX conventions* [2]. It can interact with the OS in ways web apps cannot. For example, the assistant can run PowerShell commands or manipulate system settings directly via C# and Windows APIs. This goes beyond what ChatGPT (web) or mobile assistants can do.
- **Structured Tool Calls:** We leverage the “tools” (function calling) pattern from advanced LLM design: the AI generates *structured instructions* (JSON commands) for file actions, registry edits, or app launches [9]. This allows deterministic parsing and safer execution. For example, the AI might be given a schema and produce a JSON with fields like `{"action":"open_app","program":"notepad.exe"}`. This is similar to local “scripts,” but written by the AI. (Web clients can’t run these; only a native app can take JSON and call `Process.Start()` or use Windows APIs accordingly [9].)

- **Privacy & Local Control:** All sensitive operations happen locally. User prompts and actions stay on the machine (only the text query goes out to the cloud). The app does not send user files or system info to the cloud. Like the open-source MdcAI example, we can keep all chat history in a local SQLite DB, ensuring privacy [10]. This contrasts with some SaaS bots that log and reuse prompts.
- **Offline & Failover Capability:** Having an offline mode sets this assistant apart. If the internet drops, the app still runs and can notify the user or run fallback tasks – competitors (like pure cloud agents or Copilot) simply go offline. The auto-launch on disconnect is a unique automation trigger.
- **AI-Guided Automation:** Compared to tools like Power Automate or AutoHotkey, TaskFlow AI offers a conversational interface. You *talk* to it instead of writing macros. For example, "Create a project folder for ClientX and open it in VSCode" results in those actions. This natural-language command+execute pattern is its killer feature.

Subscription & Usage Tracking

- **Subscription Model:** We build user accounts and billing via **Stripe**. The desktop app implements a login/registration UI. (The Stripe .NET SDK can be used for API calls.) Upon signup, the backend creates a Stripe Customer and Subscription for the user. In fact, developers have integrated Stripe *directly into* C# desktop apps by storing Stripe Customer IDs and even password hashes in Stripe's metadata [11] – this suggests no separate web portal is strictly needed [11]. The app will prompt the user for payment at signup (e.g. open a Stripe Checkout page in a WebView).
- **Plans & Metering:** A simple tiered plan can be used (e.g. monthly AI credits). For more granularity, Stripe's metered billing can track usage. According to Stripe docs, you can set up a "meter" for usage events – e.g. count tokens or number of API calls per month [12]. Our backend logs each AI request and reports usage to Stripe as needed. (For example, each LLM response's token count is sent as a usage event.) Stripe then bills overages automatically [12].
- **Usage Logging:** Every time the app calls the AI API, the backend logs: user ID, timestamp, token count. This serves two purposes: enforcing any rate limits, and analytics. The backend can expose an endpoint for the client app to fetch current usage stats (tokens or queries used). In a premium flow, the app could even show a small "credits remaining" indicator.
- **Implementation Note:** All network API calls from the app (chat messages, subscription validation) go through the FastAPI server. The server verifies the user's Stripe subscription status (via Stripe API/webhooks) before allowing use. This ensures only paid users or active trial users can access the AI.

Technical Challenges (and Mitigations)

- **Windows UI Learning Curve:** WinUI3 (or WPF/.NET) has a learning curve. *Mitigation:* Use XAML MVVM patterns and sample templates. Follow Microsoft's docs (e.g. use the WinUI App SDK) to get a starter app. Leverage existing WinUI component libraries to speed up development.
- **Elevated Permissions:** Some automations may need higher privileges (e.g. editing `HKLM` registry). Running the whole app as admin hurts UX. *Mitigation:* Request elevation only when needed (spawn a separate elevated process for that action). Or limit the assistant's actions to user-level tasks (document files, user settings) and warn/skip others.
- **LLM Output Uncertainty:** GPT can "hallucinate" or produce unexpected instructions. *Mitigation:* Use strict JSON schemas and system prompts so the AI must reply in the given format. Always ask for user confirmation before executing any action. Log all intended actions so the user can audit them.

- **Network Variability:** Handling intermittent connectivity while keeping UI responsive is tricky. *Mitigation:* Perform network calls asynchronously (async/await) so the UI doesn't freeze. Implement retries with exponential backoff. When offline is detected, queue API requests rather than crashing.
- **Security:** Exposing file operations to an AI is inherently risky. *Mitigation:* Sandboxing actions – for example, disallow certain commands (e.g. delete system32, or run executables not on a whitelist). Use a "dry-run" mode for dangerous commands (show the plan but do not run until confirmed). Ensure all API requests to the backend are authenticated (use HTTPS and tokens or JWTs).
- **Subscription/Auth Complexity:** Building user accounts, login UIs, password reset flows can be burdensome for a solo dev. *Mitigation:* Offload as much as possible to Stripe (which handles customer creation, email confirmations). Follow the pattern in [11] of using Stripe metadata for password hashes so that user management mostly lives on Stripe's side.
- **Resource Constraints:** A solo developer has limited time. It's critical to scope an MVP: start with basic chat + a few file actions (open folder, move file, launch app). Use open-source libraries for JSON handling, HTML rendering, etc. Avoid building a complex context-memory system (no RAG embeddings initially) – just chat history in SQLite.
- **Testing & Maintenance:** Testing the interplay of UI and system commands is complex. *Mitigation:* Develop incrementally, using unit tests for backend logic and manual tests for UI automations. Log errors robustly. Use early adopter feedback to focus on critical use-cases.

Sources: We base these design choices on industry examples and best practices. For instance, a community Windows AI app uses WebView2 for Markdown rendering [3] and stores chat locally in SQLite [10]. FastAPI is recommended for small AI backends [5], and Stripe docs outline metered billing for AI usage [12]. Windows expert blogs note using Task Scheduler for startup [7] and prefer WinUI for modern desktop apps [1]. Following these, TaskFlow AI will be a fully-native, cloud-hybrid assistant with robust offline behavior and a clear monetization path.

---

[1] Choosing the Right UI Framework for Native Windows Applications -- Visual Studio Magazine
https://visualstudiomagazine.com/articles/2024/02/13/desktop-dev.aspx

[2] [3] [4] GitHub - bawkee/MdcAi: Multipurpose Desktop Chat AI
https://github.com/bawkee/MdcAi

[5] Actions backend Nodejs or FastAPI - Plugins / Actions builders - OpenAI Developer Community
https://community.openai.com/t/actions-backend-nodejs-or-fastapi/587111

[6] Deploy FastAPI on Cloud Providers - FastAPI
https://fastapi.tiangolo.com/deployment/cloud/

[7] registry - Run C# application at Windows startup? - Stack Overflow
https://stackoverflow.com/questions/52762945/run-c-sharp-application-at-windows-startup

[8] C# How to detect internet connection / disconnection (with an event)? - Stack Overflow
https://stackoverflow.com/questions/48000031/c-sharp-how-to-detect-internet-connection-disconnection-with-an-event

[9] [10] Native Windows Desktop UI for OpenAI API - Use cases and examples - OpenAI Developer Community
https://community.openai.com/t/native-windows-desktop-ui-for-openai-api/1091928

11 Inventive, Low-Cost, Serverless Stripe Integration and the Netflix Dilemma Solved | by Tihamér Juhász | Medium
https://medium.com/@tijuhasz/inventive-low-cost-serverless-stripe-integration-and-the-netflix-dilemma-solved-f47ac2bd67e4

12 Checkout
https://docs.stripe.com/get-started/use-cases/usage-based-billing